

MoMo SMS Transactions REST API

Project Report: Security, Testing, API Documentation and DSA Results

September 29, 2026

Section	Topic
1	Security Introduction
2	Testing and Screenshots
3	API Documentation
4	DSA Results (Linear Search vs Dictionary Lookup)

Section 1: Security Introduction

For this project I protected the API with Basic Authentication. The client sends a username and password in the Authorization header, and the server decodes it and checks that they match. It is easy to set up, but it is not very secure. The main problem is that Base64 is not encryption, it is only an encoding. Anyone who can see the network traffic can copy the header and decode it back to the real password in seconds, and the password is sent again with every request. Because of this, Basic Authentication should only ever be used over HTTPS. In my `auth.py` I used a simple username and password (admin and password123) written directly in the code, which is fine for a school project but not for a real system. In real life, credentials should be kept out of the source code, for example in environment variables or a secrets manager, and passwords should be stored as salted hashes, not as plain text. Stronger alternatives are JWT (JSON Web Tokens), where the user logs in once and then sends a signed token that expires, and OAuth2, where access is given through tokens without the app ever handling the user's password.

My `is_authorized` function returns `False` for a wrong password, a wrong username, a missing header, a header that is not Basic, and text that cannot be decoded. I wrote tests for all of these cases in `test_auth.py` and they all pass.

Section 2: Testing and Screenshots

I tested the API with curl in the Ubuntu terminal while the server ran locally on port 8000. The five screenshots below show a successful GET with the correct password (200 OK), a failed GET with a wrong password (401 Unauthorized), a successful POST that created a new transaction (201 Created), a successful PUT that changed the amount from 5000 to 7500 (200 OK), and a successful DELETE (204 No Content).

#	Request	Credentials	Expected result
1	GET a transaction	Correct	200 OK
2	GET a transaction	Wrong password	401 Unauthorized
3	POST a new transaction	Correct	201 Created
4	PUT (amount 5000 to 7500)	Correct	200 OK
5	DELETE a transaction	Correct	204 No Content

```
hussaina@HUSNA: ~/momo-i x + v
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.8.10
Date: Tue, 29 Sep 2026 18:10:17 GMT
Content-Type: application/json
Content-Length: 148

{"id": 5, "transaction_type": "payment", "amount": 2000, "sender": "account_owner", "receiver": "Samuel Carter", "timestamp": "2024-05-11T18:48:49"}hussaina
hussaina@HUSNA:~/momo-rest-api-ewd$
```

Figure 1. Successful GET with the correct password returns 200 OK and the transaction record for ID 5.

```
hussaina@HUSNA: ~/momo-i x + v
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.8.10
Date: Tue, 29 Sep 2026 18:15:25 GMT
WWW-Authenticate: Basic realm="MoMo API"
Content-Type: application/json
Content-Length: 57

hussaina@HUSNA:~/momo-rest-api-ewd$ |
```

Figure 2. GET with a wrong password returns 401 Unauthorized, with a WWW-Authenticate header asking for Basic credentials.

```
hussaina@HUSNA: ~/momo-i x + v
HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.8.10
Date: Tue, 29 Sep 2026 18:16:05 GMT
Content-Type: application/json
Content-Length: 147

{"transaction_type": "payment", "amount": 5000, "sender": "account_owner", "receiver": "Test User", "timestamp": "2026-09-29T18:00:00", "id": 1692}hussaina@
hussaina@HUSNA:~/momo-rest-api-ewd$ |
```

Figure 3. Successful POST returns 201 Created; the new transaction is assigned ID 1692.

```
hussaina@HUSNA: ~/momo-i x + v
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.8.10
Date: Tue, 29 Sep 2026 18:16:35 GMT
Content-Type: application/json
Content-Length: 147

{"transaction_type": "payment", "amount": 7500, "sender": "account_owner", "receiver": "Test User", "timestamp": "2026-09-29T18:00:00", "id": 1692}hussaina@
hussaina@HUSNA:~/momo-rest-api-ewd$ |
```

Figure 4. Successful PUT returns 200 OK; the amount of transaction 1692 changes from 5000 to 7500.

```
hussaina@HUSNA: ~/momo-i x + v
HTTP/1.0 204 No Content
Server: BaseHTTP/0.6 Python/3.8.10
Date: Tue, 29 Sep 2026 18:17:03 GMT

hussaina@HUSNA:~/momo-rest-api-ewd$
```

Figure 5. Successful DELETE returns 204 No Content with an empty response body.

Section 3: API Documentation

All endpoints require the header `Authorization: Basic <credentials>`. Requests without valid credentials receive 401 Unauthorized.

Method	Endpoint	Purpose	Success
GET	/transactions	List all transactions	200 OK
GET	/transactions/{id}	Get one transaction	200 OK
POST	/transactions	Create a transaction	201 Created
PUT	/transactions/{id}	Update a transaction	200 OK
DELETE	/transactions/{id}	Delete a transaction	204 No Content

1. Get All Transactions

Endpoint	GET /transactions
Description	Retrieves a list of all parsed MoMo SMS transactions.
Headers	Authorization: Basic <credentials>

Success response (200 OK)

```
[
  {
    "id": 1,
    "transaction_type": "incoming_money",
    "amount": 2000,
    "sender": "Jane Smith",
    "receiver": "account_owner",
    "timestamp": "2024-05-10T16:30:51"
  }
]
```

Error codes

Code	Meaning
401	Unauthorized (missing or invalid credentials)

2. Get Transaction by ID

Endpoint	GET /transactions/{id}
Description	Retrieves a single transaction object by its unique sequential integer ID.
Headers	Authorization: Basic <credentials>

Success response (200 OK)

```
{
  "id": 1,
  "transaction_type": "incoming_money",
  "amount": 2000,
  "sender": "Jane Smith",
  "receiver": "account_owner",
  "timestamp": "2024-05-10T16:30:51"
}
```

Error codes

Code	Meaning
400	Bad Request (invalid ID format)
401	Unauthorized (missing or invalid credentials)
404	Not Found (transaction ID does not exist)

3. Create a Transaction

Endpoint	POST /transactions
Description	Adds a new transaction record to the system. A unique ID is auto-assigned.
Headers	Authorization: Basic <credentials> Content-Type: application/json

Request example

```
{
  "transaction_type": "transfer",
  "amount": 5000,
  "sender": "account_owner",
  "receiver": "Alex Doe",
  "timestamp": "2024-06-01T10:15:00"
}
```

Success response (201 Created)

```
{
  "id": 1692,
  "transaction_type": "transfer",
  "amount": 5000,
  "sender": "account_owner",
  "receiver": "Alex Doe",
  "timestamp": "2024-06-01T10:15:00"
}
```

Error codes

Code	Meaning
400	Bad Request (malformed JSON or missing required fields)
401	Unauthorized (missing or invalid credentials)

4. Update a Transaction

Endpoint	PUT /transactions/{id}
Description	Updates an existing transaction record by its ID. The primary ID cannot be changed.
Headers	Authorization: Basic <credentials> Content-Type: application/json

Request example

```
{
  "transaction_type": "transfer",
  "amount": 5500,
  "sender": "account_owner",
  "receiver": "Alex Doe",
  "timestamp": "2024-06-01T10:15:00"
}
```

Success response (200 OK)

```
{
  "id": 1,
  "transaction_type": "transfer",
  "amount": 5500,
  "sender": "account_owner",
  "receiver": "Alex Doe",
  "timestamp": "2024-06-01T10:15:00"
}
```

Error codes

Code	Meaning
400	Bad Request (malformed JSON, missing required fields, or invalid ID format)
401	Unauthorized (missing or invalid credentials)
404	Not Found (transaction ID does not exist)

5. Delete a Transaction

Endpoint	DELETE /transactions/{id}
Description	Removes a transaction record from the system database by its ID.
Headers	Authorization: Basic <credentials>

Success response (204 No Content)

(Empty body)

Error codes

Code	Meaning
400	Bad Request (invalid ID format)
401	Unauthorized (missing or invalid credentials)
404	Not Found (transaction ID does not exist)

Notes

A POST request must include all five fields (transaction_type, amount, sender, receiver and timestamp), otherwise the server returns 400 Bad Request. A PUT request replaces the whole record, so it also needs all five fields, and the ID in the URL is kept. IDs for new records continue from the highest existing ID, so the first record created after the 1,691 loaded ones gets ID 1692.

Section 4: DSA Results

This section compares two ways of finding a transaction by its ID: linear search and dictionary lookup. Both were timed with `time.perf_counter_ns()` on 1,691 records, using 21 record IDs spread across the dataset.

Record ID	Linear search (ns)	Dictionary lookup (ns)
1	1,800	1,700
85	4,600	800
169	17,400	500
253	78,300	1,600
337	34,900	900
421	26,600	800
505	45,800	700
589	42,900	500
673	58,200	800
757	61,800	1,300
841	40,300	600
925	28,800	600
1009	43,100	500
1093	33,400	400
1177	35,700	600
1261	48,500	500
1345	156,000	1,100
1429	104,900	6,000
1513	118,600	1,400
1597	133,800	2,300
1691	190,800	1,700
Average	62,200	1,205

Result: the average linear search took about 62,200 ns and the average dictionary lookup took about 1,205 ns, so dictionary lookup was about 51.6 times faster.

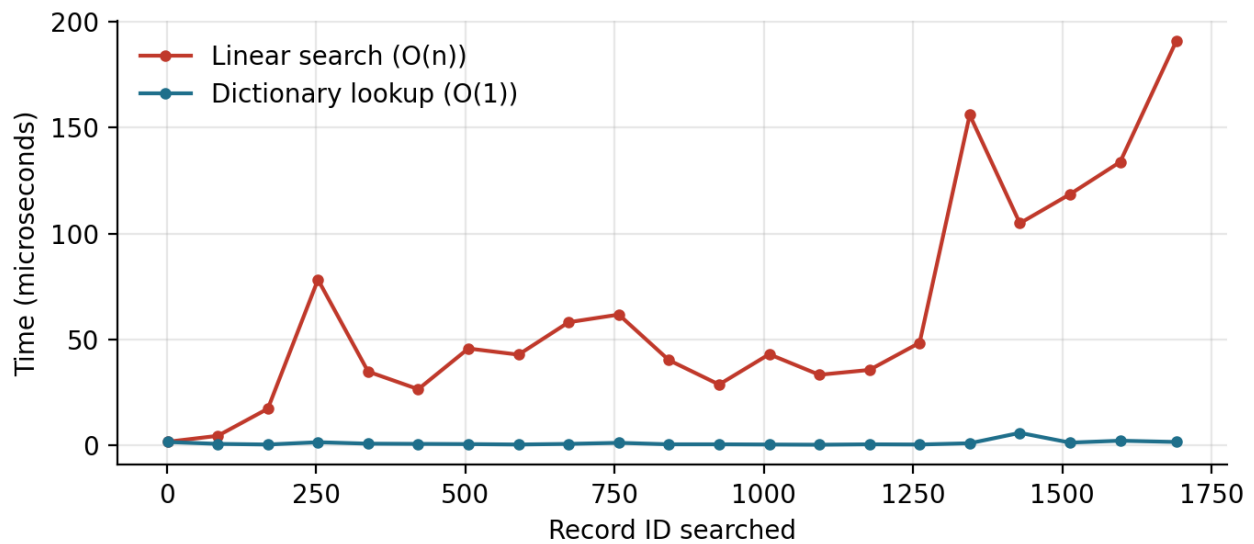


Figure 6. Time taken to find each record. Linear search gets slower for records further down the list, while dictionary lookup stays low.

Explanation

To find a transaction record by its ID, I compared two approaches on the 1,691 parsed SMS records. Linear search loops through the list one record at a time until it finds a matching ID, so in the worst case it must check every item. This makes its running time $O(n)$, and it grows as the dataset grows. Dictionary lookup first converts the list into a Python dictionary keyed by ID. A dictionary is built on a hash table, which turns the key into a position in memory, so it can jump directly to the record without checking the others. This gives $O(1)$ time, meaning the lookup cost stays roughly constant regardless of how many records there are. I timed both methods with `time.perf_counter_ns()` on 21 record IDs spread across the dataset, including the last record, which is the worst case for linear search. Linear search averaged about 62,200 ns, while dictionary lookup averaged about 1,205 ns, making it roughly 52 times faster. The results also showed that linear search became slower for records further down the list, while dictionary lookup times stayed low throughout. The trade-off is that a dictionary uses extra memory and takes a one-time cost to build, which is worthwhile when many searches are performed.